Erste Schritte in R und RStudio

Stand: 2. Oktober 2026

Eine „perfekte“ oder auch nur verbindliche Einführung in R existiert nicht. Dies zeigt gerade die Vielfalt der im Internet verfügbaren Skripte. Der in den einzelnen Veranstaltungen notwendige Programmcode wird zu den jeweiligen Terminen erläutert und vollständig zur Verfügung gestellt.

Warum verwenden wir R?

- kostenlos, open source[1], für verschiedene Betriebssysteme[2]
- zahlreiche statistische Methoden
- umfangreiche Möglichkeiten zur Datenvorbereitung und -verarbeitung
- leistungsfähig
- vielfältige Visualisierungsmöglichkeiten
- verbreiteter Einsatz in Forschung und Wirtschaft
- R ist eine vollständige[3] Programmiersprache!

R kann unter `http://cran.r-project.org` heruntergeladen werden.

Unter `http://cran.r-project.org/manuals.html` gibt es einen Überblick zahlreicher Handbücher und Tutorials zu R.

Es existieren verschiedene kostenlose Arbeitsumgebungen wie z.B. *RStudio*, welches wir in der Vorlesung und den Übungen verwenden werden. Sie können die Open-Source-Edition *RStudio IDE* von `https://posit.co/products/open-source/rstudio` herunterladen und installieren.

[1] R wird auch GNU S genannt.
[2] UNIX/Linux, Windows, MacOS
[3] Turing-vollständig

Excel? Matlab?

Excel ist weit verbreitet und leicht zu bedienen, ist im Funktionsumfang aber wesentlich eingeschränkt und kann insbesondere im Zusammenhang mit größeren Datenmengen unbrauchbar langsam werden. Es bietet nur relativ wenige statistische Methoden und nur begrenzte Visualisierungsmöglichkeiten. Daher ist es als allgemeine Statistiksoftware nicht geeignet.

Matlab bietet umfangreiche Methoden zur statistischen Auswertung und Visualisierung, welche gut dokumentiert sind. Allerdings ist *Matlab* eine kommerzielle Software. Eine Lizenz für Matlab und benötigte Bibliotheken ist üblicherweise sehr teuer.

Was kann man mit R machen?

- Rechnen
- auf Daten zugreifen
- Rohdaten zur Auswertung aufbereiten (*data wrangling*)
- Auswertungen (deskriptive und schließende Statistik)
- Modellanpassungen
- Daten und Auswertungen auf verschiedenste Art visualisieren

R-Befehle sind an die englische Sprache angelehnt. Hilfetexte und Beispiele sind entsprechend (meist) in Englisch verfasst.

Alle folgenden Erläuterungen wurden in R unters Windows getestet, sollten aber unverändert auf anderen Plattformen wie Mac OS X und Linux laufen.

Um mit R zu arbeiten, müssen wir nur RStudio starten, welches R im Hintergrund für uns aufruft.

Operatoren im ersten Überblick (vgl. unten für Details)

arithmetisch	`+ - * / %% ^`
relational	`> >= < <= == !=`
logisch	`! & |`
Modellierung	`~`
Zuweisung	`<- ->`
Listenindex (Name des Elements)	`$`
Sequenz erzeugen	`:`

Dezimalzeichen ist der Punkt und nicht das Komma, wie im Deutschen üblich. (z.B. 3.141 *statt* $3,141$*) Das Komma dient in R der Trennung von Argumenten.*

Die Arbeitsumgebung

Abfrage des Arbeitsverzeichnisses	`getwd()`
Setzen des Arbeitsverzeichnisses	`setwd("Pfad")`
Standardverzeichnis	`setwd("~")`
Was befindet sich dort?	`dir()`
Überblick über allgemeine Einstellungen	`sessionInfo()`
Liste angebundener Pakete (attached) und R-Objekte	`search()`
Objekte im Arbeitsbereich	`ls()` bzw. `objects()`
Entfernen aus dem Arbeitsbereich	`rm()`
komplett bereinigen	`rm(list=ls())` bzw. `rm(list= ls(all=TRUE))`[4]
Unterbrechen eines Befehls mit:	`<ESC>`
R-Konsole „leeren“ (*clear screen*) mit	`<CRTL>+<L>`

R ist kommandozeilenorientiert, Ausdrücke werden einfach eingegeben, z.B. `1 + 2` oder `2 ** 3`.

R-Anweisungen können über mehrere Zeilen hinweg erfolgen. Solange die Anweisung nicht syntaktisch korrekt abgeschlossen wurde, beginnt die Fortsetzungszeile mit einem automatisch eingesetzten „+“.

```
> 2 **
+
```

Mehrere Anweisungen werden mit Semikolon getrennt. Anweisungsblöcke werden mit geschweiften Klammern zusammengefasst: `{...;...;...}`.

Kommentare werden mit „`#`“ eingeleitet. Alles nachfolgende in der gleichen Zeile wird vom Befehlsinterpreter ignoriert. Kommentare können direkt neben den Ausdruck geschrieben werden.

[4] Hier schon mal vorweg: `list=` ist ein Objekt mit Elementen beliebigen Datentyps.

R als Taschenrechner

Grundlegende arithmetische Operatoren, Funktionen und Werte

Potenz	`^` oder `**`
Multiplikation, Division	`* , /`
Addition, Subtraktion	`+ , -`
ganzzahlige Division	`%/%`
Modulo Division	`%%`
Quadratwurzel	`sqrt(...)`
Betrag	`abs(...)`
Fakultät	`factorial(...)`
Exponentialfunktion	`exp(...)`
Eulersche Zahl	`exp(1)`
π	`pi`
natürlicher Logarithmus, zur Basis e	`log(...)`
dekadischer Logarithmus, zur Basis 10	`log10(...)`
binärer Logarithmus, zur Basis 2	`log2(...)`
Allgemein: Logarithmus von x zur Basis y	`logb(x, y)`
trigonometrische Funktionen	`sin(...), cos(...), tan(...), asin(...), acos(...), atan(...)`
Extremwerte (Minimum, Maximum)	`min(...), max(...)`
Rundung auf x Dezimalstellen	`round(..., x)`
nächste untere Ganzzahl	`floor(...)`
nächste obere Ganzzahl	`ceiling(...)`

```
# Beispiele:
3 ^ 4; 3 ** 4; 8 ^(1/3); sqrt(8)
116 / 13; 116 %/% 13; 116 %% 13; 783411 %% 3 == 0
factorial(6)
abs(-64.2)
round(pi); round(pi, 1); round(pi, 2); round(pi, 3)
floor(7.9); ceiling(7.9)
max(c(8, 9, 12, 24, 7, 19, 62, 54))
min(c(8, 9, 12, 24, 7, 19, 62, 54))
log(exp(1)); log(3); log10(3); log2(3); logb(9,3)
```

Die Funktion `options(digits=ZAHL)` erhöht die Ziffergenauigkeit. `ZAHL` steht für die gewünschte Ziffernanzahl, wobei die Ziffern vor dem Dezimalpunkt mitgezählt werden. Wenn die Anzahl der Ziffern vor dem Dezimalpunkt kleiner oder gleich `ZAHL` ist, werden nur alle Ziffern vor dem Dezimalpunkt angezeigt.

```
# Beispiel:
options(digits=2)
pi # liefert: 3.1

options(digits=10)
pi # liefert: 3.141592654
```

Summe und Produkt über Vektorelemente

Summe	`sum(...)`
kumulierte Summe	`cumsum(...)`
Produkt	`prod(...)`
kumuliertes Produkt	`cumprod(...)`

```
# Beispiele:
sum(2, 4, 6, 8) # alternativ: sum(c(2, 4, 6, 8))
prod(2, 4, 6, 8)

cumsum(c(2, 4, 6, 8))
cumprod(c(2, 4, 6, 8))
```

Spezielle Werte im Überblick

Infinity (Unendlichkeit)	`Inf, -Inf`
Not a Number (nicht definiert)	`NaN`
Not Available (fehlender Wert, Missing Value)	`NA`
leere Menge	`Null`

Vektoren

Es gibt neben Vektoren und Listen noch eine Reihe weiterer Objekte in R: Matrizen (matrices), Arrays, Faktoren (factors) und Datenrahmen (data frames).

In R ist alles ein Objekt. Dies gilt für Daten wie für Funktionen. Intern wird jedes Objekt durch einen Vektor repräsentiert. Den Objekten werden Werte zugewiesen. Der Zuweisungsoperator ist die Zeichenfolge „<-“ . Von der (möglichen) Benutzung des Gleichheitszeichens „=“ als Zuweisungsoperator wird abgeraten. Der Zuweisungsoperator erzeugt Objekte um Inhalte oder Zwischenergebnisse zu speichern. Namen für Objekte sollten aussagekräftig sein.

Vektoren beinhalten mehrere Elemente des gleichen Objekttyps.

Erzeugung von Vektoren

1. Zusammenfassung (engl.: Concatenation)
   `x <- c(`x_1`, ..., `x_n`)`
2. Sequenzen
   ```
   # Beispiele:
   1:3; seq(1, 4.5, length=3); seq(3,4, by=.25)
   ```
3. Wiederholungen
   ```
   # Beispiele:
   rep(3, times=4); rep(3, 4); rep(5:6, 3); rep(1:4, length=5)
   ```

Wird nur der Name des Objekts eingegeben, gibt R den Inhalt des Objekts zurück. Operationen auf einem Vektor führt R grundsätzlich komponentenweise aus.

```
# Beispiele:
# 1. Zusammenfassungen
x <- c(1,2,3,4); x; c(x,0,x)
y <- c("Ein", "kleiner", "Satz"); y; c(x,y)

# 2. Sequenzen
seq(1,10,1) # (untere Grenze, obere Grenze, Schrittweite)
seq(1,5); seq(1,10,0.1); seq(5,1,-1)
# Alternative zu seq: x <- 1:5; y <- c(1:5)

# 3. Wiederholungen
# Funktion rep(Variable, Wiederholungen)
x <- rep(0,10); rep(x,3)
x <- c(1:3); y <- c(3:1); rep(x,y)
```

Die Funktion `paste()` bietet eine weitere Möglichkeit zur Erzeugung von Vektoren.

```
# Beispiele:
paste("Ein", "kurzer", "Satz")
paste("Komponente", c(1:6))
paste(c("X","Y"), 1:10, sep="")
```

Zugriff auf Vektorelemente

Die Indizierung erfolgt über die Angabe einer Indexmenge in eckigen Klammern: `[Indexmenge]`.

```
# Beispiele:
x <- seq(10,50,10)
x          # gesamter Vektor
x[3]       # einzelnes Element (hier das dritte)
x[ c(2:4)] # einzelne Elemente (hier das zweite bis vierte)
x[-c(1,2)] # invere Indizierung[5] (hier ohne erstes und zweites Element)
```

Die Länge eines Vektors, d.h. Anzahl der Elemente, erhält man durch `length(...)`.

```
x <- c(1,2,3,4) # alternativ: x <- 1:4
length(x)

# noch mehr (Vektor-)Arithmetik:
x <- x^2
y <- x; x + y; x * y; x / y; c(x,y)^2

# und noch mehr
x <- seq(2,8,2)
x; 2 * x; 2 + x; 2^x
```

Sortieren der Komponenten eines Vektors

`sort(...)`

```
y <- c(8, 3, 5, 7, 6, 6, 8, 9, 2, 3, 9, 4, 10, 4, 11)
sort(y)      # aufsteigende Reihenfolge
rev(sort(y)) # absteigende Reihenfolge, alternativ: sort(y, dec= T)

# Ein Beispiel von M. Crawley für die Leistungsfähigkeit von R:
# Welches ist die Summe der 3 größten Werte von y ?
sum(rev(sort(y))[1:3])
```

Logik

gleich, ungleich	`==, !=`
größer als, größer gleich	`>, >=`
kleiner als, kleiner gleich	`<, <=`
nicht (Negation)	`!`
und	`&, &&`
oder	`|, ||`
entweder oder (ausschließend)	`xor()`
wahr, falsch	`TRUE, FALSE (T, F)`

[5] durch vorangestelltes Minuszeichen

```
# Beispiele:

7 != 6 + 1; 3 > 2; (5 >= 4) && (6== (5+1))

# logische Verknüpfungen
F && F && T; FALSE || TRUE

# mit logischen Werten rechnen
FALSE + TRUE + FALSE; TRUE + TRUE; F + F

x <- 1; y <- 3; x < y; 1 < 3
x > y  # größer
x == y # gleich
x != y # ungleich
x <= y # kleiner oder gleich
x >= y # größer oder gleich

# logisches "UND" und "ODER"
x <- 1; y <- 2; z <- 3; u <- 4
x < y & z < u; x < y | z < u

# Negation
x <- 1; y <- 2
z <- x < y; u <- !z

# weitere Beispiele:
x <- 1; y <- 2; z <- 3; u <- 4; (x < y) + (z < u)
x <- seq(13,97,4); sum(x[x < 10]); sum(x[x < 20]); sum(x[x < 100])
exists("x"); exists("xx")

# Zugriff auf Vektorelemente über logische Operationen
x <- c(1:7)
x[x>4]
x[x>4] <- 10 # Werte größer als 4 werden auf 10 gesetzt
```

Objekte, Datentypen und Datenstrukturen in R

Objekte bestehen letztlich aus Basisdatentypen. Die Basisdatentypen lassen sich in verschiedenen Kategorien zusammenfassen. Bei diesen Kategorien handelt es sich um: Vektoren, zusammengesetzte Datentypen, spezielle Objekte, Sprachobjekte, Funktionen und interne Datentypen. Einige Datentypen:

Datentyp	Beschreibung
NULL	leere Menge
logical	logische Werte, TRUE, FALSE
numeric	ganze und reelle Zahlen (integer, double). Beispiele: 12, 8.8
complex	komplexe Zahlen
character	Buchstaben und Zeichenfolgen (in “...“)

Abfrage und Umwandlung eines Datentyps

Abfrage:
`is.numeric()`, `is.integer()`, `is.double()`, `is.complex()`, `is.logical()`, `is.character()`, `is.na()`

Umwandlung:
`as.numeric()`, `as.integer()`, `as.double()`, `as.complex()`, `as.logical()`, `as.character()`, `as.factor()`, `as.character(12)`, `as.numeric("12")`

Bei den zusammengesetzten Datentypen besitzt der Basistyp `list()` besondere Bedeutung. Mit Blick auf die in R verfügbaren Datentypen kann leicht Verwirrung entstehen, weil neben der Abfrage des eigentlichen Typs mit `typeof()` noch der Modus `mode()` und die Klasse `class()` eines Objekts bestimmt werden können. Die Entwicklung von S und R verlief über Jahrzehnte, sodass ältere und jüngere Konzepte nebeneinander existieren. `mode()` hat seinen Ursprung in einer früheren Entwicklungsstufe. `mode()` zeigt an, wie ein Objekt gespeichert wird und gibt den physischen Datentyp wieder.

```
mode(3); mode(3.1415); mode("HTW")
mode(list("Berlin", "London", "Washington"))
mode(‘identical‘) # Bitte die Verwendung des grave accent beachten
                  # vgl. mode(’identical’)
mode(‘if‘) # Bitte die Verwendung des grave accent beachten
```

`typeof()` zeigt ebenfalls den physischen Datentyp und kann als die modernere und präzisere Variante von `mode()` aufgefasst werden.

```
typeof(3); typeof(3.1415); typeof("HTW")
typeof(list("Berlin", "London":, "Washington"))
typeof(‘identical‘) # Bitte die Verwendung des grave accent beachten
                    # vgl. typeof(’identical’)
typeof(‘if‘) # Bitte die Verwendung des grave accent beachten
```

`class()` definiert den abstrakten Typ eines Objekts. Eine Zahl kann verschiedene Dinge repräsentieren: Eine PS-Angabe oder einen Zeitpunkt (berechnet in Tagen seit dem 01. Januar 1970). In beiden Fällen sind die Angaben vom Typ *numeric*, aber das Datum besitzt die Klasse *Date*.

```
Datum <- as.Date("2015-04-13")
mode(Datum)
class(Datum)
typeof(Datum)
is.numeric(Datum)
as.numeric(Datum)
```

`class()` teilt R mit, wie ein Objekt zu interpretieren und zu verarbeiten ist.

Kategorien und Datentypen werden im Verlauf der Veranstaltung bedarfsweise näher erläutert.

Hilfe in R

RSiteSearch durchsucht das Internet nach Stichwörtern: `RSiteSearch("Boxplot")`.
Die Webseite zur Suche ist `http://rseek.org/`.

Lokale Hilfe

Der Aufruf der Hilfe zu einer Funktion mit bekanntem Namen in der R-Konsole erfolgt mit „?“.

```
# Beispiele:
?mean       # alternativ: help(mean)
?write.csv2 # alternativ: help(write.csv2)
```

Struktur einer Hilfeseite:

- Beschreibung
- Gebrauch
- Argumente
- Werte
- Referenz
- weiterführende Hinweise
- Beispiel

Hilfetexte richtig interpretieren

Unter *Usage* findet sich im Hilfetext die Darstellung des korrekten Gebrauchs einer Funktion. Es gelten folgende Konventionen:
Die Klammern enthalten die Argumente der Funktion. Argumente mit einem Gleichheitszeichen sind nicht zwingend erforderlich. Argumente ohne Gleichheitszeichen müssen angegeben werden. Bei frei wählbaren Argumente werden die Standardwerte (*default values*) angezeigt.

```
write.table(x, file = "", append = FALSE, quote = TRUE, sep = " ",
eol = "\n", na = "NA", dec = ".", row.names = TRUE, col.names = TRUE,
qmethod = c("escape", "double"), fileEncoding = "")
```

Erläuterungen zu den Argumenten finden sich im Hilfetext unter *Arguments*. `args()` liefert die Argumente und voreingestellte Werte (Defaults) einer Funktion.

Suche innerhalb der Dokumentation

`help.start()` ruft einen Browser auf und beginnt an der Wurzel der HTML-Dokumentation.

`help.search(Suchtext)` sucht nach Übereinstimmungen zwischen Suchberiff und Name, Titel, Alias-Bezeichnungen usw. Die Suche erfolgt sowohl unscharf als auch unter Berücksichtigung regulärer Ausdrücke.

```
# Beispiel
help.search("regression")
??"regression" # Kurzform, entspricht help.search("regression")
```

Suche nach einem Teilbegriff

`apropos()` liefert einen Vektor mit allen Objekten (Funktionen), die den Suchstring enthalten. Beispiel: `apropos("mean")`

Beispiel für die korrekte Verwendung einer Funktion

`example()` liefert konkrete Beispiele für die Verwendung einer Funktion.
Beispiele: `example(median); example(read.csv2)`

Für einige Funktionalitäten existieren Demos. `demo()` gibt einen Überblick über die verfügbaren Demos.

Pakete

Zahlreiche Funktionen werden in Paketen (*packages*) bereitgestellt.

```
# Überblick über installierte Pakete
installed.packages()

# Installieren und nutzen von Paketen
install.packages("MASS")
```

```
# Bereitstellung eines Pakets
library(MASS)

# Inhalt eines Pakets
library(help = "MASS")

# Die in einem Paket enthaltenen Funktionen
lsf.str("package:MASS", all = TRUE)

# Alle in einem Paket enthaltenen Objekte
# all = TRUE zeigt auch verborgene Objekte an
# (z.B. Variablen, deren Namen mit einem Punkt beginnt)
ls("package:MASS", all = TRUE)
```

Datensätze in einem Paket

```
# Liste der in einem Paket enthaltenen Datensätze mit Kurzbeschreibung
data(package = "MASS")$results

# Auf einen in einem Paket enthaltenen Datensatz zugreifen
data(Traffic, package="MASS")

# Überblick über den Datensatz gewinnen
str(Traffic)

# Objektart
class(Traffic)

# Aufbau des Datensatzes (Zeilen und Spalten)
dim(Traffic)

# Variablennamen
names(Traffic)
```

Die Bezeichnungen *library* und *package* werden häufig synonym verwendet. Tatsächlich handelt es sich um unterschiedliche Dinge.[6]

Daten einlesen

R kann Daten aus einer Vielzahl von Quellen wie CSV, Excel, Internet, Statistikpakete (SAS, SPSS, ...) und Datenbanken importieren. Einen guten Überblick bieten: `https://www.datacamp.com/doc/r/importingdata` und `http://www.r-bloggers.com/importing-data-into-r-from-different-sources/`. Wir werden hier erst einmal mit CSV-Dateien arbeiten:

[6] vgl. `http://www.r-bloggers.com/packages-v-libraries-in-r/`

```
daten <- read.csv2(file.choose(),
header= T, sep= ";", dec= ",", na.strings=c("-", "NA", "kA"))
names(daten)
dim(daten)
str(daten)
head(daten)
tail(daten)
head(daten$merkmal)
```

Abfrage von Daten direkt aus dem Internet:

```
daten <- read.table(
"http://archive.ics.uci.edu/ml/machine-learning-databases/
autos/imports-85.data", header= F, sep= ",")
```

Fehlende Werte (*missing values*)

Die Berechnung mit einem *NA*-Wert erzeugt immer *NA* als Ergebnis.

```
# Beispiele:
3 + NA; TRUE == NA; TRUE && NA
# Beachte
TRUE || NA; NA && FALSE
# weiteres Beispiel: # Wir erzeugen zunächst 50 Zufallszahlen
x <- sort(sample(100,50))
# und setzen eine Zahl auf "Missing"
x[23] <- NA
# Jetzt ziehen wir "naiv" die Summe der Zufallszahlen
sum(x) # Ergebnis "NA"
na.rm= T # entfernt die "Missings" aus der Berechnung
sum(x, na.rm=T); is.na(x)
# Der Index des fehlenden Wertes kann wie folgt bestimmt werden
which(is.na(x))
# Einen Vektor ohne "Missings" erhalten wir durch
x1 <- na.omit(x); sum(x1) # oder x2 <- na.exclude(x); sum(x2)
```

Was gibt es sonst noch?

Das Einlesen eines Skripts mit R-Anweisungen erfolgt durch `source("MeineRBefehle.R")`. Auf R in einer Veröffentlichung oder Arbeit Bezug nehmen bzw. R zitieren kann man mit `citation()`. Ein bestimmtes Paket zitieren kann man mit `citation("Paketname")`.

Der empfohlene Style Guide für die Programmierung in R ist unter folgender URL zu finden:
`https://style.tidyverse.org`.

Umfangreiche Informationen zu R und seinen Funktionen sind z.B. bei Datacamp zu finden: `https://www.datacamp.com/doc/r/category/r-documentation`.